Leilão Distribuído com WebSocket

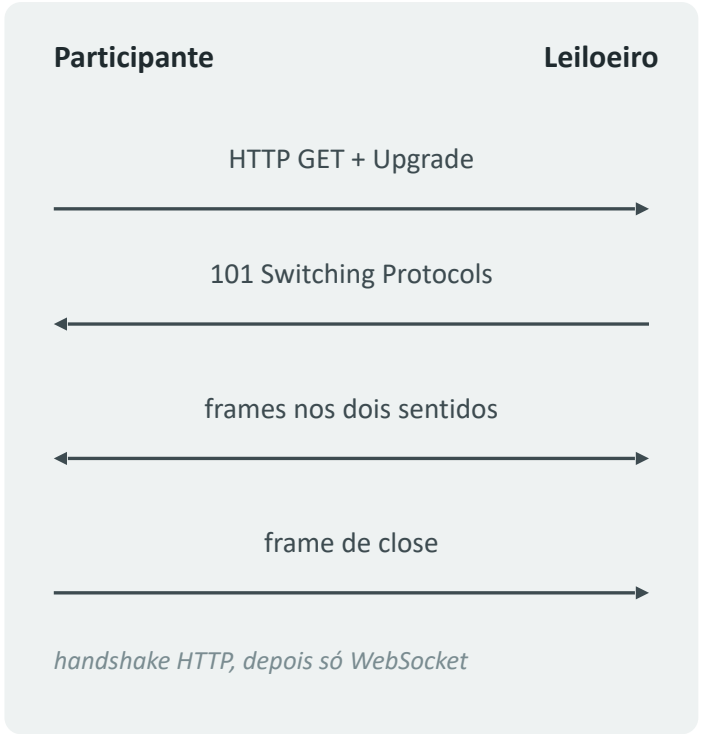
Sistemas Distribuídos e Computação Paralela · PUC-SP · Prof. Carlos Paes

Grupo 1: João Avila, Kauã Cordeiro, Luan Capella, Pedro Evaristo

Seminário 13/11/2026

WebSocket: um canal aberto nos dois sentidos

- Protocolo padronizado na RFC 6455 (2011), sobre uma única conexão TCP
- Começa como HTTP e faz upgrade: GET com "Upgrade: websocket" → resposta 101 Switching Protocols
- Full-duplex e persistente: cliente e servidor enviam quando quiserem, sem novo pedido
- Paradigma: troca de mensagens, comunicação direta entre processos
- Modelo assíncrono e orientado a eventos (open, message, close)



Vantagens, limitações e uso real

Vantagens

- Push em tempo real, sem polling
- Baixa sobrecarga por mensagem depois do handshake
- Funciona no navegador e passa por portas HTTP (80/443)

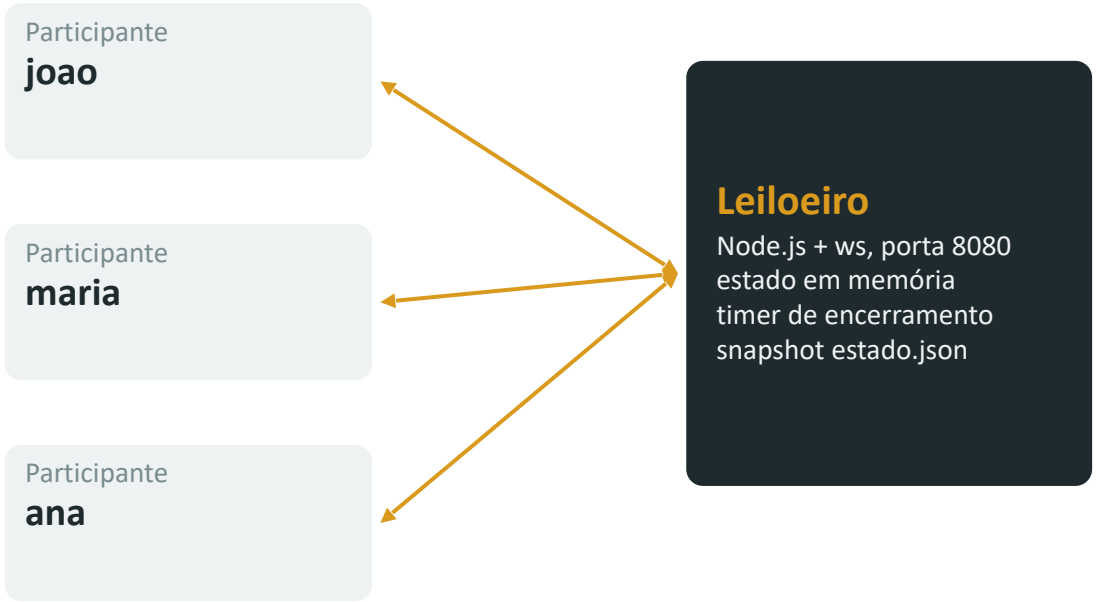
Limitações

- Sem reconexão nem garantia de entrega: fica com a aplicação
- Conexão com estado: escalar exige cuidado (sticky sessions, pub/sub)
- Sem contrato formal de mensagens (formato é por nossa conta)

Uso real

- Discord: Gateway de eventos via WebSocket
- Slack: Socket Mode para apps
- Corretoras (ex.: Coinbase): feed de cotações
- Ferramentas: ws, Socket.IO, websockets (Python)

Arquitetura da POC



Mensagens (JSON, campo "type")

Participante → Leiloeiro

`JOIN · GET_STATE · BID`

Leiloeiro → Participante

`STATE · BID_RESULT`
`NEW_HIGHEST_BID (push)`
`AUCTION_CLOSED · ERROR`

```
{"type":"BID","bidder":"joao",
 "amount":150}
```

Participante abre a conexão; depois os dois lados enviam livremente

Demonstração

1	**Leiloeiro numa máquina**	`npm run leiloeiro -- --novo`
2	**Participantes em outra**	`npm run participante -- ws://IP:8080 joao`
3	**Lance recusado e aceito**	`joao> 90` `joao> 150`
4	**Push para os outros**	maria vê "NOVO MAIOR LANCE" sem digitar
5	**Encerramento**	todos recebem o vencedor; novos lances recusados

RE2: e se o Leiloeiro cair?

Antes

joao dá 300. Snapshot gravado em estado.json antes de confirmar.

kill -9

Participantes recebem close 1006 (queda anormal) e encerram. Ninguém reconecta sozinho.

Reinício

Leiloeiro restaura 300 de joao e o mesmo horário de fim. Lance de 300 é recusado.

- Ponto único de falha: sem Leiloeiro não há leilão. Outro assumir exigiria réplica e eleição (Bully, anel)
- Lances enviados durante a queda se perdem: o protocolo não reenvia

RE3: dois lances ao mesmo tempo

5 de 5

rodadas com o mesmo valor: o primeiro a chegar ao servidor ganha, o outro recebe LOWER_THAN_CURRENT

Cenário	Resultado (5 rodadas, local)
ana 100 e bia 100	ana aceita 5×, bia recusada 5×
ana 150 e bia 160	os dois aceitos, em sequência

- Um único event loop: cada lance é tratado inteiro antes do próximo. Não há corrida nem lock
- Vale a ordem de chegada ao Leiloeiro, não o relógio de quem enviou (ligação com Lamport e NTP)
- Entre máquinas a latência pode inverter quem chega primeiro

Como usamos IA

Onde ajudou

- Roteiro da POC e formato das mensagens a partir da especificação
- Implementação do Leiloeiro, Participante e script do RE3 (Claude Code)
- Estrutura do README, do IA.md e destes slides

Onde errou e como corrigimos

- Script do RE3 travava: a resposta STATE chegava antes de o código começar a esperar por ela. Pegamos rodando o teste; corrigido esperando o STATE dentro do connect
- Participante quebrava ao fechar a entrada (readline usado depois de fechado). Corrigido com uma flag
- Todo código gerado foi testado rodando antes do commit

Conclusão

Usar WebSocket quando…

- os clientes precisam saber na hora (lances, chat, cotações)
- há muitas mensagens pequenas nos dois sentidos
- o cliente é um navegador

Evitar quando…

- consultas eventuais bastam (REST é mais simples)
- é preciso entrega garantida ou desacoplamento (MOM)
- queremos contrato formal de interface (gRPC, SOAP)

No nosso leilão apareceram: ordem dos lances decidida pela chegada ao servidor, o Leiloeiro como ponto único de falha e o acoplamento de uma conexão aberta por Participante.